

Assignment 2: Coding Basics

Jordan Heinlein

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
Sequence55 <- seq(1,55,5) #Created a sequence from 1-55 by 5 and assigned the object name "Sequence55".  
#2.  
mean(Sequence55) #Using mean() to calculate the mean of Sequence55 which is 26.
```

```
## [1] 26
```

```
median(Sequence55) #Using median() to calculate the median of Sequence55 which is 26.
```

```
## [1] 26
```

```
#3.  
mean(Sequence55) > median(Sequence55) #Providing R the conditional statement
```

```
## [1] FALSE
```

```
#that the mean of Sequence55 is greater than the median. Running this code  
#returns 'FALSE' because the mean and median values for this sequence are equal.
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
student_names <- c("Jordan","Stef","Colada","Graham") #This is a character vector.  
#Components must be in "".  
class(student_names)
```

```
## [1] "character"
```

```
student_scores <- c(90, 95, 62,100) #This is a numeric vector.  
class(student_scores)
```

```
## [1] "numeric"
```

```
student_scholarships <- c(FALSE,FALSE,TRUE,FALSE) #This is a logical vector.  
class(student_scholarships)
```

```
## [1] "logical"
```

```
student_information <- data.frame(student_names,student_scores,student_scholarships)  
#Created a data frame called "student information" by combining the student names, scores,  
#and scholarship vectors.  
  
names(student_information) <- c("Student Name", "Test Score", "Scholarship Status")  
#Updated the column names for the data frame by creating a character vector with  
#desired names.  
  
#View(student_information) View the data frame to confirm code  
#creating desired data frame.
```

9. QUESTION: How is this data frame different from a matrix?

Answer: Both data frames and matrices are two-dimensional data types, however, as is the case with this data frame, a data frame can have different classes of data (numerical, character, logical) but a matrix contains only one kind of data element (eg. numerical or characters).

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.

11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
Function1 <- function(x) {
  if(x>50) {
    "Pass"
  }
  else {
    "Fail"
  }
}
```

#11. Create a function using ifelse()

```
Function2 <- function(x) {
  ifelse(x>50,"Pass","FAIL") #Ifelse with test of x>50, if yes="Pass", else="Fail".
}
```

#12a. Run the first function with the value 52.5

```
Function1(52.5) #Returns "Pass"
```

```
## [1] "Pass"
```

#12b. Run the second function with the value 52.5

```
Function2(52.5) #Returns "Pass"
```

```
## [1] "Pass"
```

#13a. Run the first function with the vector of test scores

#Function1(student_scores) returned "Error in if (x > 50) { : the condition has length > 1"

#13b. Run the second function with the vector of test scores

```
Function2(student_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The second option using the ‘ifelse’ function worked. R vectorization means that functions will be applied to all elements of a vector at once. The “If” function did not work here because it takes only a single element as an input, and the scores vector has four elements. The ‘ifelse’ function works because it can evaluate more than one element.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)